JVM Build Tools Cheat Sheet

Ant 1.10.18 · Maven 3.10.0 (4.0 RC) · Gradle 9.8.1

Checked 2026-10-16 · Ivy 2.6.0 · Resolver Ant Tasks 2.0.0 · Maven 4.0.0-rc-7 (release candidate)

Apache Ant 1.10.18 · Java 8+ · build.xml

Command line

`ant`	run the default target of ./build.xml
`ant -f other.xml dist`	other build file, target dist
`ant -p`	list targets that have a description
`ant -Dname=value jar`	set a property (wins over the file)
`ant -propertyfile f.properties`	load properties from a file
`ant -q / -v / -d`	quiet / verbose / debug output
`ant -k`	keep going after a failed target
`ant -lib dir`	add JARs (antlibs, Ivy) to Ant's classpath
`ant -diagnostics`	Java, Ant home, optional tasks found
`ANT_OPTS / ANT_ARGS`	JVM options / default arguments

build.xml skeleton

```
<project name="tidepool" default="jar">
  <property name="build" location="build"/>
  <path id="cp"><fileset dir="lib" includes="*.jar"/></path>
  <target name="compile">
    <mkdir dir="${build}/classes"/>
    <javac srcdir="src/main/java" destdir="${build}/classes"
           release="21" classpathref="cp"
           includeantruntime="false"/>
  </target>
  <target name="jar" depends="compile" description="Build the JAR">
    <jar destfile="${build}/tidepool.jar" basedir="${build}/classes">
      <manifest><attribute name="Main-Class"
                  value="io.tidepool.Main"/></manifest>
    </jar>
  </target>
  <target name="clean"><delete dir="${build}"/></target>
</project>
```

Rules to remember

`properties`	immutable: the first definition wins
`depends="a,b"`	each target runs at most once per build
`if= / unless=`	run a target only when a property is (un)set
`<extension-point>`	a hook that other files' targets join
`<import> / <include>`	reuse files; targets overridable / prefixed
`<macrodef> / <presetdef>`	new tasks from existing ones / defaults
`<junitlauncher>`	run JUnit 5+ (JUnit Platform) tests
`<subant> / <ant>`	run targets in other build files

Dependencies: Ivy 2.6.0

```
<project xmlns:ivy="antlib:org.apache.ivy.ant">
<ivy:resolve/>                    <!-- reads ivy.xml -->
<ivy:cachepath pathid="runtime.path" conf="runtime"/>
<ivy:retrieve pattern="lib/[conf]/[artifact]-[revision].[ext]"/>
```

Dependencies: Maven Resolver Ant Tasks 2.0.0

```
<project xmlns:resolver="antlib:org.apache.maven.resolver.ant">
<resolver:resolve>
  <dependencies>
    <dependency coords="org.apache.commons:commons-csv:1.14.1"/>
  </dependencies>
  <path refid="compile.classpath" classpath="compile"/>
</resolver:resolve>
```

Apache Maven 3.10.0 · Java 8+ · pom.xml

Command line

`mvn clean verify`	build and run unit + integration tests
`mvn install / deploy`	to ~/.m2/repository / to a remote repo
`mvn -pl :engine -am verify`	one module plus the modules it needs
`mvn verify -rf :engine`	resume the reactor from a module
`mvn -T 1C verify`	parallel build, 1 thread per CPU core
`mvn -o / -U`	offline / force update of SNAPSHOTs
`mvn -B -ntp`	batch mode, no transfer progress (CI)
`mvn -P prod / -Dkey=val`	activate a profile / set a property
`mvn -DskipTests`	compile tests but do not run them
`mvn test -Dtest=HeightTest`	run one test class (Surefire)
`mvn -e / -X`	stack traces / debug logging
`mvn help:effective-pom`	the POM after inheritance and profiles
`mvn dependency:tree`	resolved dependency graph
`mvn dependency:analyze`	used-undeclared / declared-unused
`mvn wrapper:wrapper`	generate mvnw (Maven Wrapper 3.3.4)

Default lifecycle (main phases)

validate → initialize → generate-sources → process-resources → **compile** → process-test-resources → test-compile → **test** → prepare-package → **package** → pre-integration-test → integration-test → post-integration-test → **verify** → **install** → **deploy**. Other lifecycles: `clean`, `site`. Running a phase runs every phase before it.

POM skeleton

```
<project xmlns="http://maven.apache.org/POM/4.0.0">
  <modelVersion>4.0.0</modelVersion>
  <groupId>io.tidepool</groupId>
  <artifactId>tidepool-model</artifactId>
  <version>1.0.0-SNAPSHOT</version>
  <properties>
    <maven.compiler.release>21</maven.compiler.release>
  </properties>
  <dependencyManagement><dependencies>   <!-- import a BOM -->
    <dependency><groupId>org.junit</groupId><artifactId>junit-bom</artifactId>
      <version>6.1.3</version><type>pom</type><scope>import</scope></dependency>
  </dependencies></dependencyManagement>
  <dependencies>
    <dependency><groupId>org.junit.jupiter</groupId>
      <artifactId>junit-jupiter</artifactId><scope>test</scope></dependency>
  </dependencies>
</project>
```

Scopes and mediation

`compile`	default; on every classpath, transitive
`provided`	compile + test, supplied at runtime (not transitive)
`runtime / test`	runtime + test only / test only
`import`	BOM in <dependencyManagement>, type pom
`nearest wins`	shortest path, then first declaration

Maven 4 4.0.0-rc-7 · release candidate · Java 17+

`modelVersion 4.1.0`	new POM model, namespace POM/4.1.0
`<subprojects>`	replaces <modules>
`consumer POM`	flattened POM published for consumers
`before:/after: phases`	e.g. after:compile, before:all
`mvn -r verify`	resume from the last failed subproject
`mvnup -B check`	upgrade tool: check, then apply

Gradle 9.8.1 · runs on Java 17–27 · build.gradle.kts

Command line (always through the wrapper)

`./gradlew build`	assemble + check (compile, test, jar)
`./gradlew tasks --all`	list every task
`./gradlew :app:test --tests 'io.*'`	one project, filtered tests
`./gradlew build -x test`	exclude a task
`./gradlew build --dry-run`	print the task graph, run nothing
`./gradlew build --continuous`	rebuild on every file change
`--offline / --refresh-dependencies`	no network / re-check remote repos
`--rerun-tasks`	ignore up-to-date checks
`--build-cache / --configuration-cache`	reuse outputs / reuse the task graph
`--scan / --profile`	Build Scan / local HTML profile
`--warning-mode all`	show every deprecation warning
`dependencies --configuration runtimeClasspath`	dependency tree
`dependencyInsight --dependency commons-csv`	why this version
`javaToolchains / buildEnvironment`	JDKs found / build-script classpath
`gradle wrapper --gradle-version 9.8.1`	create or upgrade the wrapper

build.gradle.kts (Kotlin DSL)

```
plugins { `java-library` }
java { toolchain { languageVersion = JavaLanguageVersion.of(21) } }
dependencies {
    api(libs.commons.csv)
    testImplementation(platform(libs.junit.bom))
    testImplementation(libs.junit.jupiter)
    testRuntimeOnly(libs.junit.launcher)
}
tasks.test { useJUnitPlatform() }
tasks.register<Copy>("dist") {
    from(tasks.jar)
    into(layout.buildDirectory.dir("dist"))
}
```

gradle/libs.versions.toml

```
[versions]
junit = "6.1.3"
[libraries]
commons-csv = { module = "org.apache.commons:commons-csv", version = "1.14.1" }
junit-bom = { module = "org.junit:junit-bom", version.ref = "junit" }
junit-jupiter = { module = "org.junit.jupiter:junit-jupiter" }
```

Dependency configurations

`api`	compile + runtime, exposed to consumers (java-library)
`implementation`	compile + runtime, hidden from consumers
`compileOnly / runtimeOnly`	compile classpath only / runtime only
`testImplementation`	test compile + runtime
`platform(...) / enforcedPlatform(...)`	import a BOM / force its versions

gradle.properties

```
org.gradle.caching=true
org.gradle.configuration-cache=true
org.gradle.parallel=true
```

Phases: initialization (settings) → configuration (build scripts) → execution (tasks). Prefer `tasks.register` and lazy `Providers`.

Same job, three tools

Job	Ant (+ Ivy)	Maven	Gradle
Compile / test / package	`ant compile · ant test · ant jar (targets you write)`	`mvn compile · mvn test · mvn package`	`./gradlew compileJava · test · jar (or assemble)`
Full build with checks	`ant (default target)`	`mvn verify`	`./gradlew build`
Install to local repository	`ivy:publish to a local resolver / resolver:install`	`mvn install`	`./gradlew publishToMavenLocal (maven-publish)`
Show dependencies	`ivy:report`	`mvn dependency:tree`	`./gradlew dependencies`
Pin the tool version	`– (install Ant 1.10.18)`	`mvnw (mvn wrapper:wrapper)`	`gradlew (gradle wrapper --gradle-version 9.8.1)`

Each command and snippet is explained, with run output, on the matching page of the JVM Build Tools reference.